

P1.1 $|R(s, a)| \leq R_{\max} \quad \forall s \in \mathcal{S} \text{ \& } a \in \mathcal{A} \quad \text{--- (1)}$

$$G_i^s = \sum_{t=0}^{|\tau_i^s|} \gamma^t R(s_{t,i}^s, A_{t,i}^s) \quad \text{--- (2)} \quad 0 \leq \gamma < 1$$

↓
returns

↓ state ↓ action

at time t & trajectory τ_i^s

ask. is to find α & β $\left\{ \alpha_i \leq G_i^s \leq \beta_i \right\}$

let $R = R_{\max}$ for greatest & $R = -R_{\max}$ where R is least

$$-R_{\max} \leq R(s_{t,i}^s, A_{t,i}^s) \leq R_{\max} \quad \text{--- (3)}$$

③ $\times \gamma$ both sides

$$-\gamma^t R_{\max} \leq \gamma^t R(s_{t,i}^s, A_{t,i}^s) \leq \gamma^t R_{\max}$$

next summing over the entire trajectory 1

$$-\sum_{t=0}^{|\tau_i^s|} \gamma^t R_{\max} \leq \underbrace{\sum_{t=0}^{|\tau_i^s|} \gamma^t R(s_{t,i}^s, A_{t,i}^s)}_{G_i^s} \leq \sum_{t=0}^{|\tau_i^s|} \gamma^t R_{\max}$$

$$\therefore -R_{\max} \sum_{t=0}^{|\tau_i^s|} \gamma^t \leq G_i^s \leq R_{\max} \sum_{t=0}^{|\tau_i^s|} \gamma^t$$

$$\therefore -R_{\max} \frac{1 - \gamma^{|\tau_i^s|+1}}{1 - \gamma} \leq G_i^s \leq R_{\max} \frac{1 - \gamma^{|\tau_i^s|+1}}{1 - \gamma}$$

$$\therefore \alpha_i = -R_{\max} \frac{1 - \gamma^{|\tau_i^s|+1}}{1 - \gamma} \quad \beta_i = +R_{\max} \frac{1 - \gamma^{|\tau_i^s|+1}}{1 - \gamma} \quad \left. \vphantom{\alpha_i} \right\} \text{Ans}$$

$$0 \leq \gamma < 1 \text{ \& } \gamma^{|\tau_i^s|+1} \leq 1 \text{ numerically } \therefore \frac{1 - \gamma^{|\tau_i^s|+1}}{1 - \gamma} \leq \frac{1}{1 - \gamma}$$

$$\therefore \alpha_i = -\frac{R_{\max}}{1 - \gamma}, \quad \beta_i = \frac{R_{\max}}{1 - \gamma} \quad \left. \vphantom{\alpha_i} \right\} \text{Answer}$$

(2)

P1.2 $E(s) = \sum_{i=1}^{N^s} G_i^s$ sum of all sample returns.

Given,

$$\mathbb{E} [G_i^s] = v^\pi(s)$$

$$A_t \sim \pi(\cdot | s_t)$$

$$s_{t+1} \sim P(\cdot | s_t, A_t)$$

$$\mathbb{E}_{\substack{A_t \sim \pi(\cdot | s_t) \\ s_{t+1} \sim P(\cdot | s_t, A_t)}} [E(s)] = \mathbb{E}_{\substack{A_t \sim \pi(\cdot | s_t) \\ s_{t+1} \sim P(\cdot | s_t, A_t)}} \left[\sum_{i=1}^{N^s} G_i^s \right] = \sum_{i=1}^{N^s} \mathbb{E}_{\substack{A_t \sim \pi(\cdot | s_t) \\ s_{t+1} \sim P(\cdot | s_t, A_t)}} [G_i^s] \quad \text{--- (1)}$$

By definition, we get $\mathbb{E}[G_i^s] = v^\pi(s)$
state val of π under policy
in discounted episodic setting

$\therefore$ (1) becomes

$$\mathbb{E}_{\substack{A_t \sim \pi(\cdot | s_t) \\ s_{t+1} \sim P(\cdot | s_t, A_t)}} [E(s)] = \sum_{i=1}^{N^s} \underbrace{v^\pi(s)}_{\text{Answer}} = N^s v^\pi(s)$$

P1.3 Apply Hoeffding's inequality to

$\mathbb{P}(|E(s) - \mathbb{E}(E(s))| \geq \epsilon)$, since G_i^s are independent let

$a \leq G_i^s \leq b$; bounded

Hoeffding inequality eqn would be

$$\mathbb{P}\left(\left|\sum_{i=1}^{N^s} G_i^s - \mathbb{E}[G_i^s]\right| \geq \epsilon\right) \leq 2 \exp\left(-\frac{2\epsilon^2}{\sum_{i=1}^{N^s} (b_i - a_i)^2}\right)$$

$$\text{Since } \mathbb{E}[E(s)] = \sum_{i=1}^{N^s} \mathbb{E}[G_i^s]$$

$$\Rightarrow \mathbb{P}(|E(s) - \mathbb{E}[E(s)]| \geq \epsilon) \leq 2 \exp\left(-\frac{2\epsilon^2}{\sum_{i=1}^{N^s} (b_i - a_i)^2}\right)$$

from prob 1.1

$$\alpha_i = -R_{\max} \frac{1 - \gamma^{|\tau_i^s|+1}}{1 - \gamma}$$

$$\beta_i = R_{\max} \frac{1 - \gamma^{|\tau_i^s|+1}}{1 - \gamma}$$

$$\therefore \beta_i - \alpha_i = \frac{2R_{\max}}{1 - \gamma} (1 - \gamma^{|\tau_i^s|+1})$$

$$\therefore \mathbb{P}(|E(s) - \mathbb{E}[E(s)]| \geq \epsilon) \leq 2 \exp \left(- \frac{2\epsilon^2(1-\gamma)^2}{4R_{\max}^2 \sum_{i=1}^{N^s} (1-\gamma^{|z_i^s|+1})^2} \right) \quad (3)$$

Ans

P1.4 $\hat{V}^\pi(s) = \frac{1}{N^s} E(s)$ given

also from P.1.2 $\mathbb{E}[E(s)] = N^s V^\pi(s)$
 $A_t \sim \pi(\cdot | s_t)$
 $s_t \sim P(\cdot | s_t, A_t)$

$$\therefore |\hat{V}^\pi(s) - V^\pi(s)| = \frac{1}{N^s} |E(s) - \mathbb{E}[E(s)]|$$

$A_t \sim \pi(\cdot | s_t)$
 $s_t \sim P(\cdot | s_t, A_t)$

$$\mathbb{P}(|\hat{V}^\pi(s) - V^\pi(s)| \geq \epsilon') = \mathbb{P}(|E(s) - \mathbb{E}[E(s)]| \geq N^s \epsilon') \quad \epsilon' > 0$$

Hoeffding

$$\mathbb{P}(|E(s) - \mathbb{E}[E(s)]| \geq \epsilon) \leq 2 \exp \left(- \frac{2\epsilon^2}{\sum_{i=1}^{N^s} (\beta_i - \alpha_i)^2} \right)$$

$\epsilon = N^s \epsilon'$

$$\text{or } \mathbb{P}(|\hat{V}^\pi(s) - V^\pi(s)| \geq \epsilon') \leq 2 \exp \left(- \frac{2(N^s)^2 (\epsilon')^2}{\sum_{i=1}^{N^s} (\beta_i - \alpha_i)^2} \right)$$

from P1.1 $\beta_i - \alpha_i = \frac{2R_{\max}}{1-\gamma} (1-\gamma^{|z_i^s|+1})$

$$\therefore \mathbb{P}(|\hat{V}^\pi(s) - V^\pi(s)| \geq \epsilon') \leq 2 \exp \left(- \frac{2(N^s)^2 (1-\gamma)^2}{4R_{\max}^2 \sum_{i=1}^{N^s} (1-\gamma^{|z_i^s|+1})^2} \right)$$

from P1.3 also, $\beta_i - \alpha_i = \frac{2R_{\max}}{1-\gamma}$
 2 P.1.1

Ans

$$\begin{aligned} 0 &\leq \gamma < 1 \\ \gamma^{|z_i^s|+1} &\leq 1 \\ \Rightarrow \alpha_i &= -\frac{R_{\max}}{1-\gamma} \\ \beta_i &= \frac{R_{\max}}{1-\gamma} \end{aligned}$$

$$\begin{aligned} \therefore \sum_{i=1}^{N^s} (\beta_i - \alpha_i)^2 &= N^s \left(\frac{2R_{\max}}{1-\gamma} \right)^2 \\ &= \frac{4R_{\max}^2}{(1-\gamma)^2} N^s \end{aligned}$$

$$\therefore \mathbb{P}(|\hat{V}^\pi(s) - V^\pi(s)| \geq \epsilon') \leq 2 \exp \left(- \frac{(1-\gamma)^2 N^s (\epsilon')^2}{2R_{\max}^2} \right) \quad \underline{\underline{\text{Ans}}}$$

Simplified

P1.5

(4)

$$\|\hat{V}^\pi - V^\pi\|_\infty = \max_{s \in \mathcal{S}} |\hat{V}^\pi(s) - V^\pi(s)|$$

$$\begin{aligned} \mathbb{P}(\|\hat{V}^\pi - V^\pi\|_\infty \geq \epsilon') &= \mathbb{P}\left(\max_{s \in \mathcal{S}} |\hat{V}^\pi(s) - V^\pi(s)| \geq \epsilon'\right) \\ &= \mathbb{P}\left(\bigcup_{s \in \mathcal{S}} \{|\hat{V}^\pi(s) - V^\pi(s)| \geq \epsilon'\}\right) \\ &\leq \sum_{s \in \mathcal{S}} \mathbb{P}(|\hat{V}^\pi(s) - V^\pi(s)| \geq \epsilon') \end{aligned}$$

from Hoeffding result from P.1.4.

$$\sum_{s \in \mathcal{S}} \mathbb{P}(|\hat{V}^\pi(s) - V^\pi(s)| \geq \epsilon') \leq \sum_{i=1}^{|\mathcal{S}|} 2 \exp\left(-\frac{N_s^s \epsilon'^2 (1-\gamma)^2}{2 R_{\max}^2}\right)$$

Simplified form

$$\text{or } \sum_{s \in \mathcal{S}} \mathbb{P}(|\hat{V}^\pi(s) - V^\pi(s)| \geq \epsilon') \leq \sum_{i=1}^{|\mathcal{S}|} 2 \exp\left(-\frac{N^s (\epsilon')^2 (1-\gamma)^2}{2 R_{\max}^2 (1-\gamma^{|\mathcal{I}_i^s|+1})^2}\right)$$

General form.

$$\leq 2|\mathcal{S}| \exp\left(-\frac{(\epsilon')^2 N^s (1-\gamma)^2}{2 R_{\max}^2 (1-\gamma^{|\mathcal{I}_i^s|+1})^2}\right)$$

$\forall \epsilon' > 0$

$$\text{or } \leq 2|\mathcal{S}| \exp\left(-\frac{(1-\gamma)^2 N^s (\epsilon')^2}{2 R_{\max}^2}\right)$$

Simplified.

Answer

P2.

Lecture 12

$$G = \sum_{t=t_{s,a}}^{|\mathcal{T}|} \gamma^{t-t_{s,a}} R_t$$

for visit to (s, a) at $t_{s,a}$ in an episode τ

$$\therefore G = R(s_{t_{s,a}}, a_{t_{s,a}}) + \gamma R(s_{t_{s,a}+1}, a_{t_{s,a}+1}) + \dots$$

In problem table for time t visit with return G_t with reward r_t

$$G_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots$$

τ_1 : visit $t_{b,x} = 1$ with rewards $(r_1, r_2) = (-1, 2)$

$$G_{11} = r_1 + \gamma r_2 = -1 + 2\gamma$$

τ_2 : visit $t_{b,x} = 0$ rewards $r_0 = -1, r_1 = 2, r_2 = 0, r_3 = -1, r_4 = 1.5, r_5 = 1.5$

$$G_{10} = r_0 + \gamma r_1 + \gamma^2 r_2 + \gamma^3 r_3 + \gamma^4 r_4 + \gamma^5 r_5$$

$$G_{10} = -1 + \gamma(2) + \gamma^2(0) + \gamma^3(-1) + \gamma^4(1.5) + \gamma^5(1.5)$$

$$G_{10} = -1 + 2\gamma - \gamma^3 + 1.5\gamma^4 + 1.5\gamma^5$$

τ_3 : visit $t_{b,x} = 3$

$$G_{13} = r_3 + \gamma r_4 + \gamma^2 r_5$$

$$= -1 + \gamma(1.5) + \gamma^2(1.5)$$

$$= -1 + 1.5\gamma + 1.5\gamma^2$$

Total visits $N(b, x) = 3$

Average returns

$$Q^{\pi}(b, x) = \frac{1}{3} [(-1 + 2\gamma) + (-1 + 2\gamma - \gamma^3 + 1.5\gamma^4 + 1.5\gamma^5) + (-1 + 1.5\gamma + 1.5\gamma^2)]$$

$$= \frac{1}{3} [-3 + 5.5\gamma + 1.5\gamma^2 - \gamma^3 + 1.5\gamma^4 + 1.5\gamma^5]$$

$$Q^{\pi}(b, x) = -1 + \frac{5.5}{3}\gamma + \frac{1.5}{3}\gamma^2 - \frac{1}{3}\gamma^3 + \frac{1.5}{3}\gamma^4 + \frac{1.5}{3}\gamma^5$$

$$= -1 + 1.83\gamma + 0.5\gamma^2 - 0.33\gamma^3 + 0.5\gamma^4 + 0.5\gamma^5$$

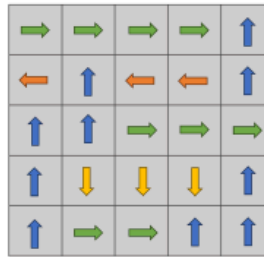
—X—

Problem 3: We want to evaluate a policy for the same example in Problem 3 of Homework 3 in a model-free setting, i.e., when the environment model (the transition function and the reward function) is unknown to the agent. Consider an MDP over the grid-world illustrated below, where the goal is to take the agent to the treasure chest while avoiding the lightning.



- The state space contains the cells of the grid-world.
- The action space is {up, down, left, right}.
- The agent can only move to its adjacent cells, i.e., the cells that are above, below, to the left, or to the right of its current cell. If the agent is not at the boundary cells, each action will take the agent to the expected cell with probability 0.85 and to one of the remaining three cells, each with probability 0.05. If the agent is at the boundary, it will remain at its current cell with the sum of probabilities that would have taken it outside the grid-world. The cell with a mountain cannot be accessed, i.e., if adjacent to the mountain, the agent will remain at its current cell with the probability that would have taken it to the mountain. All actions in the cell with the lightning bolt, the one with the treasure chest, and the ones with a mountain will keep the agent in its current cell.
- The agent receives a reward of 0 in every cell for all actions except two cells. If at the cell with a lightning bolt, it will receive a reward of -1 for all actions, and if at the cell with the treasure chest, it will receive a reward of $+1$ for all actions.
- The discount factor is 0.95.
- Model the setting as an episodic one, where each episode ends once the agent is at one of the terminal states, i.e., the cell with the lightning bolt, the one with the treasure chest, and the ones with a mountain. Once the agent reaches those terminal states, consider that the agent takes one action in those states, receives its reward value, and then, the episode terminates. Once an episode ends, the state resets to (agent respawns at) an initial state in the next episode.

Evaluate the deterministic, stationary policy shown in the figure below, where each arrow represents the prescribed action in each cell. This is the same policy evaluated in Homework 3; however, here we assume that the model is unknown to the agent and the setting is episodic. *Notice that for running the model-free algorithms, there should still be an environment model that can generate the samples. For instance, you may have an environment function in your code that takes a state, along with an action, and outputs a next state sampled according to the true (unknown to the agent) transition probabilities and a reward according to the true (unknown to the agent) reward function.*



1. Run a first-visit Monte Carlo algorithm to evaluate this policy. Describe how you ensure that each state is visited often so that its value is estimated accurately. Pick a termination criteria and justify it. Report the value function at all states after the algorithm terminates.
2. Run a one-step temporal-difference learning algorithm to evaluate this policy. Describe how you ensure that each state is visited often so that its value is estimated accurately. Pick a termination criteria and justify it. Report the value function at all states after the algorithm terminates.
3. Compute the true value function and report it at all states.
[Hint: You can apply the analytical solution for policy evaluation but it requires some modifications since this is an episodic setting and not an infinite-horizon setting. To view the problem as an infinite-horizon one and apply the analytical solution, you may consider an extra (fictitious) state where any actions in the terminal states takes the state to this fictitious one. All actions in this new state will keep the agent in the same state and have a reward of 0.]
4. Plot the sequence of errors in the value function with respect to the number of episodes for both algorithms (Monte Carlo and temporal-difference learning) in the same figure. In particular, plot $\|V_n - V^\pi\|_2$ against $n \in \mathbb{N}_0$, where V_n is the value function after n episodes and V^π is the true value function computed in Part 3.

[Note: For this problem, attach all your code in a programming language of your choice to the end of your submission. You may get help from or use the existing implementations of the Monte Carlo and the temporal-difference learning algorithms.]

Answer3.1: First-Visit MC policy evaluation (episodic, model-free)

```
In [1]: # From Homework3 setup of gridworld
        """
        Gridworld policy evaluation

        MDP:
        - 5x5 grid (x=0 to 4 left→right, y=0 to 4 bottom→top), agent start at (0,0)
        - Actions: U,D,L,R. Intended = 0.85, each other = 0.05
        - Mountains, Lightning, Treasure are absorbing
        - Rewards: -1 on Lightning, +1 on Treasure, 0 elsewhere
        - Discount gamma = 0.95

        Outputs:
        - Value function V_pi (per-state and as a 5x5 grid)
        - J(pi) = V_pi(start)
        """

        import numpy as np
        from typing import Dict, Tuple, List
        import matplotlib.pyplot as plt
```

```

# Problem spec
W, H = 5, 5
GAMMA = 0.95
EPSILON = 0.0 # Use for  $\epsilon$ -greedy exploration

# Cell types
N, M, L, T = "N", "M", "L", "T"

# Board layout (x,y) with (0,0) bottom-left
MOUNTAINS = {(2, 1), (3, 1), (1, 3)}
LIGHTNING = (2, 3)
TREASURE = (4, 4)

# Policy
policy_cols = [
    "URRUU",
    "UDDDU",
    "UURRR",
    "LULLU",
    "RRRRU",
]
policy_grid = np.array([list(col) for col in policy_cols]).T # shape (5,5)

# Actions
ACTIONS: Dict[str, Tuple[int, int]] = {
    "U": (0, 1),
    "D": (0, -1),
    "L": (-1, 0),
    "R": (1, 0),
}
DIRECTIONS = ["U", "D", "L", "R"]
P_INTENDED, P_OTHER = 0.85, 0.05
START = (0, 0)

# Helper funcs
def s_idx(x: int, y: int) -> int:
    return y * W + x
def inv_idx(s: int) -> Tuple[int, int]:
    return (s % W, s // W)
def in_bounds(x: int, y: int) -> bool:
    return 0 <= x < W and 0 <= y < H

def build_board() -> np.ndarray:
    board = np.full((W, H), N, dtype="<U1")
    for (x, y) in MOUNTAINS: board[x, y] = M
    board[LIGHTNING] = L; board[TREASURE] = T
    return board

def print_values(v: np.ndarray):
    print("Value function V_pi (per state)")
    for s, vv in enumerate(v):
        print(f"s={s:2d}: {vv: .10f}")
    print("\n V_pi as 5x5 grid (y=top→bottom rows)")
    grid = np.zeros((H, W))
    for s, vv in enumerate(v):

```

```

    x, y = inv_idx(s)
    grid[H - 1 - y, x] = vv
    for row in grid:
        print("[" + " ".join(f"{x:10.3f}" for x in row) + "]")

# Build P_pi and r_pi
def build_mdp_matrices(policy: np.ndarray, gamma: float = GAMMA):
    board = build_board()
    absorbing = set(MOUNTAINS) | {LIGHTNING, TREASURE}
    nS = W * H
    P = np.zeros((nS, nS)); r = np.zeros(nS)
    for x in range(W):
        for y in range(H):
            s = s_idx(x, y)
            if (x, y) == LIGHTNING: r[s] = -1.0
            elif (x, y) == TREASURE: r[s] = 1.0
            if (x, y) in absorbing:
                P[s, s] = 1.0
                continue
            a = policy[x, y]
            probs = {d: P_OTHER for d in DIRECTIONS}
            probs[a] = P_INTENDED
            for d, p in probs.items():
                dx, dy = ACTIONS[d]
                nx, ny = x + dx, y + dy
                if not in_bounds(nx, ny) or board[nx, ny] == M:
                    P[s, s] += p
                else:
                    P[s, s_idx(nx, ny)] += p
    return P, r, board

# Environment simulator (model-free)
rng = np.random.default_rng(42)
board = build_board()
absorbing = set(MOUNTAINS) | {LIGHTNING, TREASURE}
all_states = [(x, y) for x in range(W) for y in range(H)]
nonterminal_states = [(x, y) for (x, y) in all_states if (x, y) not in absorbing]

def reward(s):
    return -1.0 if s == LIGHTNING else (1.0 if s == TREASURE else 0.0)

def env_step(s, a):
    x, y = s
    if s in absorbing:
        return s, reward(s), True
    probs = {d: P_OTHER for d in DIRECTIONS}
    probs[a] = P_INTENDED
    dirs = np.array(DIRECTIONS)
    outcome = rng.choice(dirs, p=np.array([probs[d] for d in dirs]))
    dx, dy = ACTIONS[outcome]
    nx, ny = x + dx, y + dy
    if not in_bounds(nx, ny) or (nx, ny) in MOUNTAINS:
        nx, ny = x, y
    return (nx, ny), reward((x, y)), False

def epsilon_greedy_action(s, eps=EPSILON):

```

```

    if rng.random() < eps:
        return rng.choice(DIRECTIONS)
    return policy_grid[s[0], s[1]]

# Monte Carlo (First-Visit)
def generate_episode():
    s = nonterminal_states[rng.integers(len(nonterminal_states))]
    episode = []
    steps = 0
    while True:
        a = epsilon_greedy_action(s, eps=0.0)
        s_next, r, done = env_step(s, a)
        episode.append((s, r))
        if done or steps > 2000:
            break
        s = s_next
        steps += 1
    return episode

gamma = GAMMA
nS = W * H
v_mc = np.zeros(nS)
counts_first = np.zeros(nS, dtype=int)
recent_deltas = []
episodes = 0

def s_idx_xy(s): return s[1]*W + s[0]

while True:
    ep = generate_episode()
    episodes += 1
    G = 0.0
    seen = set()
    for t in range(len(ep) - 1, -1, -1):
        s_t, r_t = ep[t]
        G = r_t + gamma * G
        if s_t not in seen:
            seen.add(s_t)
            si = s_idx_xy(s_t)
            counts_first[si] += 1
            old = v_mc[si]
            v_mc[si] += (G - v_mc[si]) / counts_first[si]
            recent_deltas.append(abs(v_mc[si] - old))
    if len(recent_deltas) > 50:
        recent_deltas = recent_deltas[-50:]
    covered = all(counts_first[s_idx_xy(s)] >= 200 for s in nonterminal_states)
    stable = (len(recent_deltas) == 50) and (max(recent_deltas) <= 0.01)
    if covered and stable:
        break

print("\n-----Answer to Homework4 Part3.1 below-----")
print("First-Visit Monte Carlo - V_pi estimates (episodic):")
print_values(v_mc)
print(f"\nJ_hat (start @ {START}) = {v_mc[s_idx(*START)].:10f}")

```

-----Answer to Homework4 Part3.1 below-----

First-Visit Monte Carlo – V_pi estimates (episodic):

Value function V_pi (per state)

```
s= 0: 0.2063234174
s= 1: 0.1391843955
s= 2: 0.1957888635
s= 3: 0.2953430805
s= 4: 0.3349924309
s= 5: 0.2314331420
s= 6: 0.1219186625
s= 7: 0.0000000000
s= 8: 0.0000000000
s= 9: 0.4660170395
s=10: 0.3599778350
s=11: 0.2106457319
s=12: 0.2234901370
s=13: 0.3997587295
s=14: 0.7343340082
s=15: 0.5794754870
s=16: 0.0000000000
s=17: -1.0000000000
s=18: -0.7340395162
s=19: 0.8486077815
s=20: 0.6224121605
s=21: 0.6667402658
s=22: 0.7105826151
s=23: 0.8481884665
s=24: 1.0000000000
```

V_pi as 5x5 grid (y=top→bottom rows)

```
[ 0.622 0.667 0.711 0.848 1.000]
[ 0.579 0.000 -1.000 -0.734 0.849]
[ 0.360 0.211 0.223 0.400 0.734]
[ 0.231 0.122 0.000 0.000 0.466]
[ 0.206 0.139 0.196 0.295 0.335]
```

J_hat (start @ (0, 0)) = 0.2063234174

Coverage of visits & Termination for First-Visit Monte Carlo (Q3.1)

Ensuring frequent visits to every state.

Uniform random starts Each episode begins from a state sampled uniformly among all non-terminal cells (excluding Mountains, Lightning, and Treasure). This guarantees that every non-terminal state is selected as a start and first-visited many times, independent of policy reachability.

ε-greedy exploration Inside each episode, the action is chosen via function

`epsilon_greedy_action` see code line 133 onward where `EPSILON = 0.0` . Optionally stochastic exploration if epsilon is chosen say 0.1 or so.

Step cap Episodes are capped at 2000 steps to avoid unbounded rollouts. This limits variance in episode length.

Tracked coverage metric. A state first-visit counter ($N(s)$) used such that a minimum visitation quota is kept: $[\min_{s \text{ non-terminal}} N(s); \geq; N_{\min}, \quad N_{\min} = 200.]$

Termination criterion and justification The algorithm terminates when both visitation coverage and stability are achieved:

$$[\text{Coverage: } \min_{s \text{ non-terminal}} N(s) \geq N_{\min}, \quad \text{Stability: } \max_{\text{last } W_\varepsilon \text{ updates}} \Delta \leq \varepsilon,]$$

where (Δ) is the maximum absolute value change in any state after an episode, ($W_\varepsilon = 50$) is the moving window length, and ($\varepsilon = 0.01$).

($\varepsilon = 0.01$) corresponds to $\sim 1\%$ of the max possible return magnitude ($|G| \leq 1$).
 ($N_{\min} = 200$) balances computational cost and estimator variance. Together, these provide a practical operationalization of "0.01-accurate" Monte Carlo estimates for every state. We get $J_\pi(MC) = V_\pi(0, 0) \sim 0.25$, which is close to and true analytical result ~ 0.22

Answer3.2: one-step temporal-difference learning algorithm to evaluate this policy.
 (episodic, model-free)

```
In [2]: # TD(0) Policy Evaluation
v_td = np.zeros(nS)
visits = np.zeros(nS, dtype=int)
episodes = 0
stable_cnt = 0
v_prev = v_td.copy()

def td0_episode():
    s = nonterminal_states[rng.integers(len(nonterminal_states))]
    steps = 0
    while True:
        a = epsilon_greedy_action(s, eps=0.0)
        s_next, r, done = env_step(s, a)
        si = s_idx_xy(s)
        visits[si] += 1
        alpha = 0.1

        target = r if done else r + gamma * v_td[s_idx_xy(s_next)]
        v_td[si] += alpha * (target - v_td[si])
        if done or steps > 2000:
            break
        s = s_next
        steps += 1
    return steps

while True:
    nsteps = td0_episode()
    episodes += 1
    delta_inf = np.max(np.abs(v_td - v_prev))
    v_prev[:] = v_td
    stable_cnt = stable_cnt + 1 if delta_inf <= 0.01 else 0
```

```

covered = all(visits[s_idx(x,y)] >= 300 for (x,y) in nonterminal_states)
stable = (stable_cnt >= 50)
if (covered and stable) or (episodes >= 20000):
    break

print("\n----- Answer to HW4 3.2: TD(0) policy evaluation -----")
print_values(v_td)
print(f"\nJ_hat^TD (start @ {START}) = {v_td[s_idx(*START)]:.10f}")

```

----- Answer to HW4 3.2: TD(0) policy evaluation -----

Value function V_{π} (per state)

```

s= 0: 0.2428382069
s= 1: 0.1458131052
s= 2: 0.1551611803
s= 3: 0.1915479031
s= 4: 0.3104354089
s= 5: 0.2590123876
s= 6: 0.1366807466
s= 7: 0.0000000000
s= 8: 0.0000000000
s= 9: 0.3504232603
s=10: 0.3029477863
s=11: 0.1637509906
s=12: 0.2435687867
s=13: 0.2008982836
s=14: 0.3501830038
s=15: 0.3235545815
s=16: 0.0000000000
s=17: -1.0000000000
s=18: -0.7721661453
s=19: 0.7763301632
s=20: 0.6844633683
s=21: 0.7199984383
s=22: 0.6446397604
s=23: 0.8846689122
s=24: 1.0000000000

```

V_{π} as 5x5 grid (y=top→bottom rows)

```

[ 0.684 0.720 0.645 0.885 1.000]
[ 0.324 0.000 -1.000 -0.772 0.776]
[ 0.303 0.164 0.244 0.201 0.350]
[ 0.259 0.137 0.000 0.000 0.350]
[ 0.243 0.146 0.155 0.192 0.310]

```

$J_{\text{hat}}^{\text{TD}}$ (start @ (0, 0)) = 0.2428382069

Ensuring frequent visits (coverage) To guarantee that each state is visited often, each episode begins from a state sampled uniformly over all non-terminal cells (excluding Mountains, Lightning, and Treasure). This ensures repeated visits to states that may be hard to reach from (0,0) under the fixed policy. Since $TD(0)$ updates on every visit, a per-state visit counter $N(s)$ is tracked and require

$$\min_{s \in \mathcal{S}_{\text{nonterm}}} N(s) \geq N_{\min} \quad (\text{used } N_{\min} = 300).$$

This quota directly reduces variance of the value estimates by enforcing sufficient sampling at each state.

Next, the algorithm terminated when both of the following hold:

$$\min_{s \in \mathcal{S}_{\text{nonterm}}} N(s) \geq N_{\min},$$

$$\max_s |V_{t+1}(s) - V_t(s)| \leq \varepsilon \quad \text{for } W_\varepsilon \text{ consecutive episodes,}$$

with $\varepsilon = 0.01$ and $W_\varepsilon = 50$.

In this episodic setting, each return is a single terminal reward in $[-1, 1]$, so an ℓ_∞ tolerance of $\varepsilon = 0.01$ targets $\sim 1\%$ accuracy. The coverage condition controls estimator variance by ensuring sufficient updates per state, while the stability condition indicates practical convergence of the value estimates. We get $J_\pi(TD) = V_\pi(0, 0) \sim 0.2123$, which is close to MC result and analytical result

Answer 3.3: Computation of true episodic value function

```
In [3]: # True episodic V^pi

TERMINALS = set(MOUNTAINS) | {LIGHTNING, TREASURE}
nS = W * H
F = nS
r_aug = np.zeros(nS + 1)
for x in range(W):
    for y in range(H):
        s = s_idx(x, y)
        if (x, y) == LIGHTNING:
            r_aug[s] = -1.0
        elif (x, y) == TREASURE:
            r_aug[s] = 1.0
P_aug = np.zeros((nS + 1, nS + 1))
for x in range(W):
    for y in range(H):
        s = s_idx(x, y)
        if (x, y) in TERMINALS:
            P_aug[s, F] = 1.0
            continue
        a = policy_grid[x, y]
        probs = {d: P_OTHER for d in DIRECTIONS}
        probs[a] = P_INTENDED
        for d, p in probs.items():
            dx, dy = ACTIONS[d]
            nx, ny = x + dx, y + dy
            if not in_bounds(nx, ny) or (nx, ny) in MOUNTAINS:
                P_aug[s, s] += p
            else:
                P_aug[s, s_idx(nx, ny)] += p
P_aug[F, F] = 1.0
A = np.eye(nS + 1) - GAMMA * P_aug
v_aug = np.linalg.solve(A, r_aug)
v_true = v_aug[:nS]
```

```
print("\n----- Answer to HW4 3.3:True episodic V^pi via analytical solution -----")
print_values(v_true)
print(f"J_true (start @ {START}) = {v_true[s_idx(*START)].:10f}")
```

----- Answer to HW4 3.3:True episodic V^pi via analytical solution -----

Value function V_pi (per state)

```
s= 0: 0.2169482146
s= 1: 0.1320637818
s= 2: 0.1353218528
s= 3: 0.1438925662
s= 4: 0.3039291386
s= 5: 0.2353747425
s= 6: 0.1308066487
s= 7: 0.0000000000
s= 8: 0.0000000000
s= 9: 0.3321621964
s=10: 0.2571840078
s=11: 0.1425585006
s=12: 0.1897464250
s=13: 0.2742562737
s=14: 0.3543902831
s=15: 0.2811342890
s=16: 0.0000000000
s=17: -1.0000000000
s=18: -0.7148212225
s=19: 0.8297947825
s=20: 0.6010154007
s=21: 0.6570465125
s=22: 0.7010264549
s=23: 0.8470811008
s=24: 1.0000000000
```

V_pi as 5x5 grid (y=top→bottom rows)

```
[ 0.601  0.657  0.701  0.847  1.000]
[ 0.281  0.000 -1.000 -0.715  0.830]
[ 0.257  0.143  0.190  0.274  0.354]
[ 0.235  0.131  0.000  0.000  0.332]
[ 0.217  0.132  0.135  0.144  0.304]
```

J_true (start @ (0, 0)) = 0.2169482146

Problem 3.4 Plot the sequence of errors in the value function with respect to the number of episodes for both algorithms

In [4]: *# Part 3.4 – Compare Monte Carlo vs TD(0): L2 error vs episodes*

```
EPISODES = 10_000
alpha = 0.1
EPSILON = 0

rng = np.random.default_rng(42)
v_mc = np.zeros(nS)
counts_first = np.zeros(nS, dtype=int)
err_mc = []
```

```

for _ in range(EPIISODES):
    s = nonterminal_states[rng.integers(len(nonterminal_states))]
    traj = []
    steps = 0
    while True:
        a = epsilon_greedy_action(s, eps=EPSILON)
        s_next, r, done = env_step(s, a)
        traj.append((s, r))
        if done or steps > 2000:
            break
        s = s_next
        steps += 1

    # First-visit MC update
    G = 0.0
    seen = set()
    for t in range(len(traj)-1, -1, -1):
        st, rt = traj[t]
        G = rt + GAMMA * G
        if st not in seen:
            seen.add(st)
            i = s_idx_xy(st)
            counts_first[i] += 1
            v_mc[i] += (G - v_mc[i]) / counts_first[i]

    # Track error
    err_mc.append(np.linalg.norm(v_mc - v_true, ord=2))

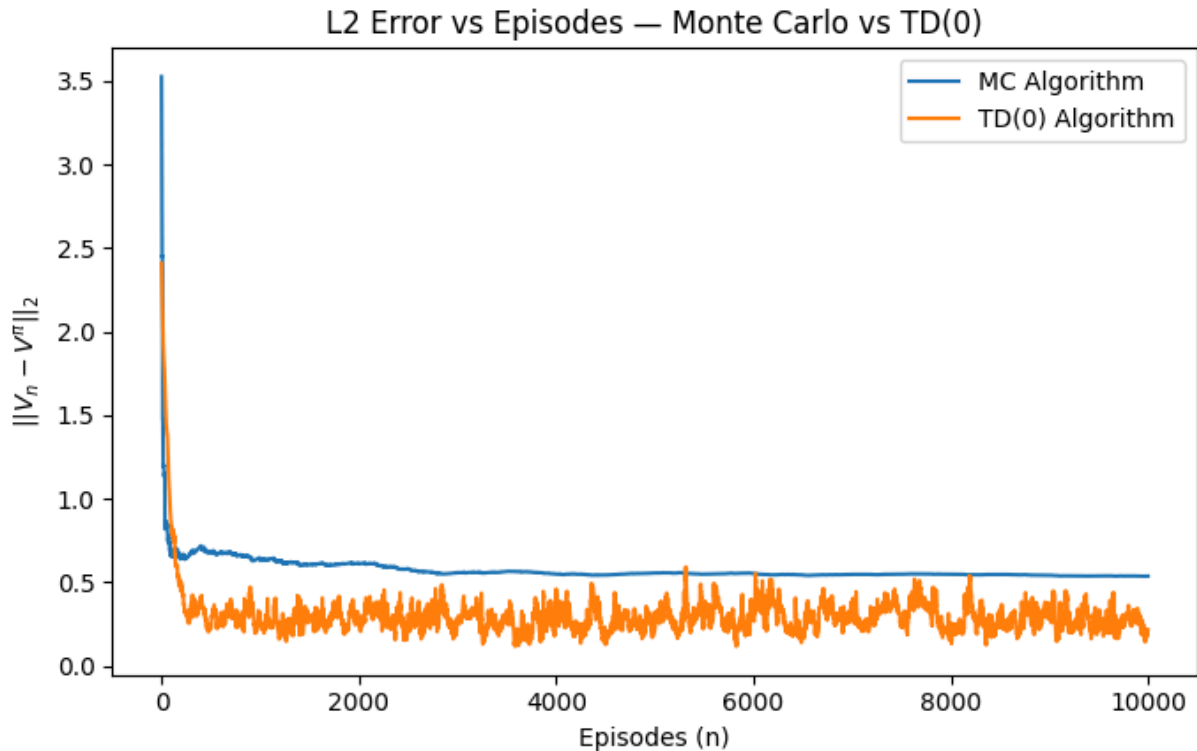
# TD
rng = np.random.default_rng(42)
v_td = np.zeros(nS)
err_td = []

for _ in range(EPIISODES):
    s = nonterminal_states[rng.integers(len(nonterminal_states))]
    steps = 0
    while True:
        a = epsilon_greedy_action(s, eps=EPSILON)
        s_next, r, done = env_step(s, a)
        i = s_idx_xy(s)
        target = r if done else r + GAMMA * v_td[s_idx_xy(s_next)]
        v_td[i] += alpha * (target - v_td[i])
        if done or steps > 2000:
            break
        s = s_next
        steps += 1
    err_td.append(np.linalg.norm(v_td - v_true, ord=2))

# Plot error vs episode
plt.figure(figsize=(7,4.5))
plt.plot(err_mc, label='MC Algorithm')
plt.plot(err_td, label='TD(0) Algorithm')
plt.xlabel('Episodes (n)')
plt.ylabel(r'$||V_n - V^{\pi}||_2$')
plt.title('L2 Error vs Episodes - Monte Carlo vs TD(0)')

```

```
plt.legend()
plt.grid(False)
plt.tight_layout()
plt.show()
```



This may not be part of homework4. Explored `EPSILON = 0.1` and run the L2 error code again.

btw, setting $\epsilon = 0$ means there is no built in exploration but rather randomness of states and the robot follows the given policy, so Monte Carlo and TD(0) evaluate the true target V^π . With $\epsilon = 0.1$, the agent follows the policy only 90% of the time and acts randomly 10%, which changes the policy to π^ϵ . This improves state visitation.

In [5]: *# Part 3.4 – Compare Monte Carlo vs TD(0): L2 error vs episodes*

```
EPISODES = 10_000
alpha = 0.1
EPSILON = 0.1

rng = np.random.default_rng(42)
v_mc = np.zeros(nS)
counts_first = np.zeros(nS, dtype=int)
err_mc = []

for _ in range(EPISODES):
    s = nonterminal_states[rng.integers(len(nonterminal_states))]
    traj = []
    steps = 0
    while True:
        a = epsilon_greedy_action(s, eps=EPSILON)
```

```

s_next, r, done = env_step(s, a)
traj.append((s, r))
if done or steps > 2000:
    break
s = s_next
steps += 1

# First-visit MC update
G = 0.0
seen = set()
for t in range(len(traj)-1, -1, -1):
    st, rt = traj[t]
    G = rt + GAMMA * G
    if st not in seen:
        seen.add(st)
        i = s_idx_xy(st)
        counts_first[i] += 1
        v_mc[i] += (G - v_mc[i]) / counts_first[i]

# Track error
err_mc.append(np.linalg.norm(v_mc - v_true, ord=2))

# save  $\epsilon=0$  results
err_mc_eps0 = err_mc.copy()
err_td_eps0 = err_td.copy()

# TD
rng = np.random.default_rng(42)
v_td = np.zeros(nS)
err_td = []

for _ in range(EPISODES):
    s = nonterminal_states[rng.integers(len(nonterminal_states))]
    steps = 0
    while True:
        a = epsilon_greedy_action(s, eps=EPSILON)
        s_next, r, done = env_step(s, a)
        i = s_idx_xy(s)
        target = r if done else r + GAMMA * v_td[s_idx_xy(s_next)]
        v_td[i] += alpha * (target - v_td[i])
        if done or steps > 2000:
            break
        s = s_next
        steps += 1
    # Track error
    err_td.append(np.linalg.norm(v_td - v_true, ord=2))

# save  $\epsilon=0.1$  results
err_mc_eps01 = err_mc.copy()
err_td_eps01 = err_td.copy()

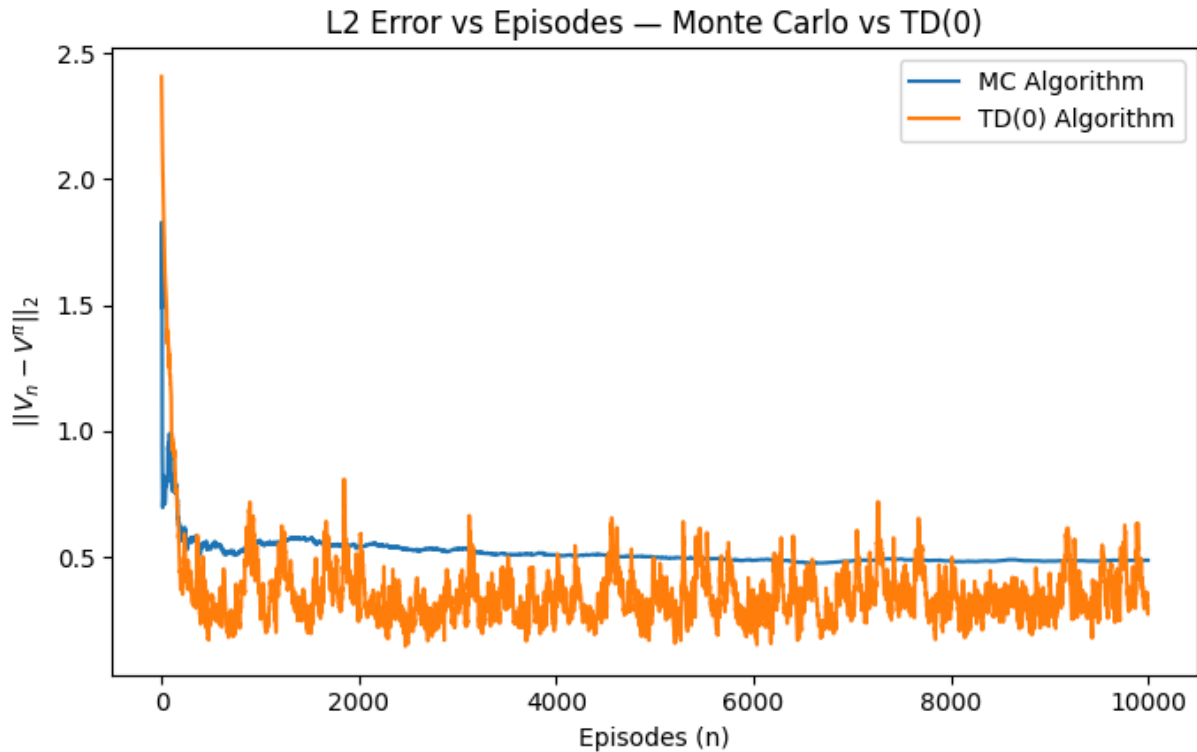
# Plot error vs episode
plt.figure(figsize=(7,4.5))
plt.plot(err_mc, label='MC Algorithm')
plt.plot(err_td, label='TD(0) Algorithm')

```

```

plt.xlabel('Episodes (n)')
plt.ylabel(r'$||V_n - V^\pi||_2$')
plt.title('L2 Error vs Episodes – Monte Carlo vs TD(0)')
plt.legend()
plt.grid(False)
plt.tight_layout()
plt.show()

```



In [6]: *# Quantitative summary*

```

def summarize_two(err):
    return {
        "final_error": float(err[-1]),
        "area under curve": float(np.trapezoid(err, dx=1))
    }

results = {
    "MC  $\epsilon=0$ ": summarize_two(err_mc_eps0),
    "TD  $\epsilon=0$ ": summarize_two(err_td_eps0),
    "MC  $\epsilon=0.1$ ": summarize_two(err_mc_eps01),
    "TD  $\epsilon=0.1$ ": summarize_two(err_td_eps01),
}

for k, v in results.items():
    print(k, v)

```

```

MC  $\epsilon=0$  {'final_error': 0.48812709425620143, 'area under curve': 5152.448857516514}
TD  $\epsilon=0$  {'final_error': 0.21409850073405376, 'area under curve': 2975.8955020783687}
MC  $\epsilon=0.1$  {'final_error': 0.48812709425620143, 'area under curve': 5152.448857516514}
TD  $\epsilon=0.1$  {'final_error': 0.30327844153165145, 'area under curve': 3573.665695545143}

```

FIN